

NeuraTrade — Guia Teórico

A teoria por trás de cada decisão: finanças, estatística e redes neurais explicadas do zero

Projeto NeuraTrade — Redes Neurais (2026.1)

23 de junho de 2026

Resumo

Este documento é um **guia autocontido**: ele explica, partindo de quase nenhum pré-requisito, toda a teoria que sustenta o projeto NeuraTrade — a detecção não supervisionada de anomalias em ações da B3 com um *LSTM-Autoencoder*. A cada conceito apresentamos a *intuição*, um *exemplo* (no domínio do projeto ou fora dele), um trecho de *código* quando ajuda, e a caixa “**No NeuraTrade**” ligando a teoria à decisão concreta tomada. Ao final, um mapa liga cada decisão ao seu registro (ADR) e há um glossário. Não é preciso ler o código do projeto para entender este texto.

Sumário

1	O problema, em uma frase	3
1.1	O caso concreto do NeuraTrade	3
1.2	Como ler este guia	3
2	Fundamentos de finanças: do preço ao retorno	3
2.1	Preço não é a melhor variável	3
2.2	Retorno: a variação relativa	3
2.3	Log-retorno: a versão que usamos	4
2.4	Volatilidade: o “nível de agitação”	4
3	Fundamentos de estatística e probabilidade	4
3.1	Distribuição, média e desvio-padrão	4
3.2	Percentil: cortando pela frequência, não pela fórmula	5
3.3	Normalização: colocar tudo na mesma escala	5
4	Treino, teste e o pecado do vazamento temporal	5
4.1	Por que dividir os dados	5
4.2	O vazamento (<i>data leakage</i>) temporal	6
4.3	A regra de ouro: dividir <i>antes</i> , ajustar só no treino	6
5	Redes neurais do zero	6
5.1	O neurônio artificial	6
5.2	Camadas e a rede densa (MLP)	7
5.3	Função de perda: medir o erro	7
5.4	Como a rede aprende: gradiente descendente	7
5.5	Overfitting, validação e <i>early stopping</i>	7

6	Séries temporais e redes recorrentes (LSTM)	8
6.1	Por que uma MLP comum não basta	8
6.2	Redes recorrentes (RNN)	8
6.3	LSTM: memória com portões	8
6.4	Janelas deslizantes: fatiando o tempo	8
7	Autoencoder: aprender a normalidade comprimindo	9
7.1	A ideia: comprimir e reconstruir	9
7.2	Por que isso detecta anomalias	9
7.3	A arquitetura do NeuraTrade	9
8	Deteção: do erro ao alarme	10
8.1	Erro de reconstrução por janela	10
8.2	Limiar estático: uma linha fixa	10
8.3	Limiar dinâmico: uma linha que se adapta	10
9	Avaliação: como saber se está funcionando	11
9.1	O dilema: não temos gabarito	11
9.2	Injeção sintética de anomalias	11
9.3	As métricas: precisão, revocação e F1	11
9.4	Duas armadilhas que aprendemos na prática	12
10	Validação narrativa e comparação setorial	13
10.1	Cruzar com a história	13
10.2	Os dois testes que isso permite	13
11	Mapa: decisão → teoria → registro	14
11.1	O fio condutor	14
12	Fase 2: três aprimoramentos e sua teoria	14
12.1	Agregar o erro: média esconde o pico	14
12.2	Validação que respeita o tempo: <i>walk-forward</i>	15
12.3	Mais de uma variável: entrada multivariada (OHLCV)	16
12.4	Fecho da Fase 2 (M9)	16
13	Glossário	17

1 O problema, em uma frase

Queremos que um computador aprenda sozinho como é o comportamento “normal” do preço de uma ação e, depois, aponte os dias em que esse comportamento foge do normal — sem que ninguém precise dizer de antemão quais dias são “estranhos”.

Isso é *detecção não supervisionada de anomalias*. “Não supervisionada” porque não temos uma lista de respostas certas (quais dias são anômalos); o modelo precisa inferir a normalidade apenas observando os dados.

Intuição 1.1. Pense em alguém que ouve uma orquestra todos os dias por dez anos. Sem nunca ter recebido uma “lista de erros”, essa pessoa passa a estranhar imediatamente quando um instrumento desafina — porque aprendeu, por exposição, como soa o normal. Nosso modelo faz o mesmo com preços de ações.

1.1 O caso concreto do NeuraTrade

Estudamos quatro ações da bolsa brasileira (B3), uma por setor: **PETR4** (Petrobras, energia), **VALE3** (Vale, mineração), **AMER3** (Americanas, varejo) e **ITUB4** (Itaú, financeiro). Treinamos um modelo *por ação* usando os anos de 2010 a 2019 como “período normal”, e depois pedimos a ele que examine 2020 a 2024 e sinalize os trechos anômalos. Por fim, conferimos se os trechos sinalizados coincidem com eventos reais (crise da COVID, fraude da Americanas, etc.).

1.2 Como ler este guia

A ordem é de baixo para cima: primeiro os tijolos (finanças, estatística, redes neurais), depois como eles se montam no modelo (LSTM-Autoencoder) e, enfim, como detectamos e avaliamos anomalias. Três tipos de caixa aparecem ao longo do texto:

- **Intuição** — a ideia em linguagem comum, sem fórmulas.
- **Exemplo** — um caso numérico pequeno, às vezes fora de finanças.
- **No NeuraTrade** — a ponte entre a teoria e a decisão real do projeto.

Os trechos de código são em Python e servem para tornar a ideia concreta; entendê-los não é obrigatório para acompanhar o raciocínio.

2 Fundamentos de finanças: do preço ao retorno

2.1 Preço não é a melhor variável

O dado bruto de uma ação é o **preço de fechamento** de cada dia. Mas o preço, sozinho, é um número difícil de comparar: uma ação que custa R\$50 e sobe R\$1 variou muito menos, proporcionalmente, do que uma de R\$5 que sobe R\$1. Além disso, preços tendem a crescer ao longo dos anos (inflação, crescimento), o que confunde um modelo que busca padrões estáveis.

2.2 Retorno: a variação relativa

Definição 2.1 (Retorno simples). O retorno simples do dia t é a variação percentual do preço:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} = \frac{P_t}{P_{t-1}} - 1,$$

onde P_t é o preço de fechamento no dia t .

2.3 Log-retorno: a versão que usamos

Na prática trabalhamos com o **log-retorno**, o logaritmo natural da razão entre preços consecutivos:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right).$$

Intuição 2.2. Para variações pequenas (o caso quase sempre), $r_t \approx R_t$: uma alta de 1% dá $r_t \approx 0,00995$. A vantagem do logaritmo é matemática e prática:

- **Somam-se no tempo.** O log-retorno de uma semana é a soma dos log-retornos diários. Com retornos simples teríamos que multiplicar $(1 + R_1)(1 + R_2) \dots$, bem menos cômodo.
- **Simetria.** Cair de 100 para 50 e voltar de 50 para 100 dá log-retornos $-0,69$ e $+0,69$ (opostos exatos); com retorno simples seriam -50% e $+100\%$, assimétricos.
- **Imune a fatores multiplicativos** como desdobramentos de ações (*splits*), porque o log transforma multiplicação em soma.

Exemplo 2.3. Preço passa de $P_{t-1} = 20,00$ para $P_t = 20,40$. Retorno simples: $20,40/20,00 - 1 = 0,02$ (2%). Log-retorno: $\ln(20,40/20,00) = \ln(1,02) \approx 0,0198$. Quase iguais.

```
1 import numpy as np
2 # precos: serie diaria de fechamento (P_t)
3 log_ret = np.log(precos / precos.shift(1)).dropna()
```

Listing 1: Log-retorno em uma linha com pandas/numpy.

2.4 Volatilidade: o “nível de agitação”

Definição 2.4 (Volatilidade). Volatilidade é o quanto os retornos oscilam. Mede-se, em geral, pelo *desvio-padrão* dos retornos (Seção 3): retornos muito espalhados $\Rightarrow$ alta volatilidade; retornos quase constantes $\Rightarrow$ baixa volatilidade.

Mercados alternam **regimes**: períodos calmos (baixa volatilidade) e períodos turbulentos (alta volatilidade, tipicamente em crises). Esse conceito será central: uma anomalia é, grosso modo, um comportamento incompatível com o regime recente.

No NeuraTrade. Usamos o **log-retorno diário do preço de fechamento** como única variável de entrada do modelo. A escolha está registrada como decisão de projeto: é invariante a *splits* e estaciona a série (remove a tendência de crescimento do preço), facilitando o aprendizado do que é “normal”. Uma consequência importante (Seção 4) é que o período de treino 2010–2019, embora chamado de “normal”, contém crises reais (recessão de 2014–2016, Lava Jato): “normal” aqui é *relativo ao que o modelo viu*.

3 Fundamentos de estatística e probabilidade

Para decidir o que é “fora do normal” precisamos descrever o normal com números. Estatística é a linguagem disso.

3.1 Distribuição, média e desvio-padrão

Definição 3.1 (Distribuição). A distribuição de um conjunto de valores diz *quais valores aparecem e com que frequência*. Visualiza-se com um **histograma**: barras cuja altura é a contagem de valores em cada faixa.

Definição 3.2 (Média e desvio-padrão). A **média** $\mu = \frac{1}{n} \sum_i x_i$ é o centro de massa dos dados. O **desvio-padrão** $\sigma = \sqrt{\frac{1}{n} \sum_i (x_i - \mu)^2}$ mede o espalhamento típico em torno da média: pequeno σ = valores agrupados, grande σ = valores dispersos.

Intuição 3.3. As alturas de pessoas adultas têm média $\approx 1,70$ m e desvio-padrão $\approx 0,10$ m. Alguém com 1,72 m é banal; com 2,10 m é *raro* — está a quatro desvios-padrão da média. “Quantos desvios-padrão de distância” é uma medida natural de “quão incomum”.

3.2 Percentil: cortando pela frequência, não pela fórmula

Definição 3.4 (Percentil). O percentil p é o valor abaixo do qual caem $p\%$ das observações. O percentil 95 (p95) é o valor que deixa 95% dos dados abaixo e 5% acima.

Exemplo 3.5. Em 100 provas com notas ordenadas, o p95 é (aproximadamente) a 95ª nota: só 5 alunos tiraram acima dela. Quem passa do p95 está no topo dos 5%.

Intuição 3.6 (Por que percentil e não “média + k desvios?”). A regra “ $\mu + k\sigma$ ” pressupõe que os dados seguem o sino da curva normal. Retornos financeiros *não* seguem: têm **caudas pesadas** (eventos extremos são mais comuns do que a curva normal preveria). O percentil é **não paramétrico** — não assume formato nenhum, apenas conta frequências — por isso é mais robusto aqui.

No NeuraTrade. O limiar que separa “normal” de “anômalo” (Seção 8) é o **percentil 95 do erro de reconstrução**: marcamos como suspeitos os 5% de maior erro. Isso embute uma *taxa-base* de $\approx 5\%$ de marcações mesmo em dados normais — um detalhe que volta a importar na avaliação (Seção 9).

3.3 Normalização: colocar tudo na mesma escala

Redes neurais aprendem melhor quando os números de entrada vivem em uma faixa pequena e comparável (tipicamente $[0, 1]$). A **normalização Min–Max** faz isso linearmente:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

O menor valor vira 0, o maior vira 1, e o resto fica proporcional no meio.

```
1 from sklearn.preprocessing import MinMaxScaler
2 scaler = MinMaxScaler()
3 scaler.fit(treino.reshape(-1, 1))      # aprende min e max -- so do treino!
4 treino_norm = scaler.transform(treino.reshape(-1, 1))
5 teste_norm = scaler.transform(teste.reshape(-1, 1))
```

Listing 2: Min–Max com scikit-learn.

Intuição 3.7. O detalhe crucial está no comentário do código: o `scaler` aprende $x_{\min}, x_{\max}$ **apenas no treino**. Por quê? Porque usar o mínimo e o máximo do período de teste seria espiar o futuro. Esse é o tema da próxima seção.

4 Treino, teste e o pecado do vazamento temporal

4.1 Por que dividir os dados

Para saber se um modelo *aprendeu* (e não apenas *decorou*), guardamos uma parte dos dados que ele nunca vê no treino e a usamos só para avaliar. Em problemas comuns, essa divisão é aleatória. Em **séries temporais**, não pode ser.

4.2 O vazamento (*data leakage*) temporal

Definição 4.1 (Vazamento temporal). Acontece quando informação do futuro influencia, ainda que indiretamente, o que o modelo aprende sobre o passado. O modelo parece ótimo na avaliação, mas fracassa no uso real — porque, na realidade, o futuro não está disponível.

Intuição 4.2. Imagine estudar para uma prova com um gabarito que, por engano, inclui as questões da prova de amanhã. Sua nota no simulado será excelente e completamente enganosa. Embaralhar datas de uma série temporal faz exatamente isso: coloca dias de 2024 no material de estudo de um modelo que deveria conhecer só até 2019.

4.3 A regra de ouro: dividir *antes*, ajustar só no treino

A ordem correta é:

1. **Dividir no tempo:** tudo até uma data de corte é treino; o que vem depois é teste. Sem embaralhar.
2. **Ajustar a normalização só no treino** (aprender $x_{\min}, x_{\max}$ apenas com o passado) e então *aplicar* essa mesma transformação ao teste.

Se invertêssemos — normalizar com todos os dados e só depois dividir — os extremos de 2020–2024 (crash da COVID!) influenciariam a escala usada no treino. Vazamento.

```
1 # 1) split TEMPORAL: treino ate 2019, teste depois
2 treino = serie[serie.index <= "2019-12-31"]
3 teste = serie[serie.index > "2019-12-31"]
4
5 # 2) scaler aprende SD no treino, depois aplica nos dois
6 scaler = MinMaxScaler().fit(treino.values.reshape(-1, 1))
7 treino_norm = scaler.transform(treino.values.reshape(-1, 1))
8 teste_norm = scaler.transform(teste.values.reshape(-1, 1))
```

Listing 3: Split temporal antes do scaler (resumo do pipeline do projeto).

Intuição 4.3 (Um efeito colateral saudável). Como a escala vem só do treino, valores de teste podem cair *fora* de $[0, 1]$ — por exemplo, um crash maior que tudo visto em 2010–2019. Isso não é bug: é justamente o sinal de algo fora do normal, exatamente o que queremos detectar.

No NeuraTrade. O corte é **2019-12-31**: treino 2010–2019, teste 2020–2024. O `MinMaxScaler` é ajustado só no treino. Esse cuidado é tão central que virou a primeira decisão registrada do projeto. Mas há uma limitação honesta: o período de treino contém crises (recessão 2014–2016, Lava Jato, *impeachment* de 2016), então o modelo aprende parte da turbulência como “normal” — um viés conservador que tende a *subestimar* anomalias. Assumimos e relatamos essa limitação em vez de mascará-la.

5 Redes neurais do zero

5.1 O neurônio artificial

Definição 5.1 (Neurônio). Um neurônio recebe números de entrada $x_1, \dots, x_n$, multiplica cada um por um **peso** w_i , soma tudo com um **viés** b , e passa o resultado por uma **função de ativação** f :

$$y = f(\sum_i w_i x_i + b).$$

Intuição 5.2. Os pesos dizem “quanto cada entrada importa”. Treinar a rede é, no fundo, achar os pesos que fazem a saída ser útil. A ativação f introduz *não-linearidade*: sem ela, empilhar neurônios daria sempre uma simples combinação linear, incapaz de representar padrões complexos. Uma ativação muito usada é a **ReLU**, $f(z) = \max(0, z)$: deixa passar o positivo, zera o negativo.

5.2 Camadas e a rede densa (MLP)

Empilhando neurônios em **camadas** — a saída de uma alimenta a próxima — obtemos uma rede. Quando cada neurônio se liga a todos da camada seguinte, é uma camada **densa** (*fully connected*); a rede toda é um *Multilayer Perceptron* (MLP).

```
1 from tensorflow import keras
2 modelo = keras.Sequential([
3     keras.layers.Dense(8, activation="relu", input_shape=(3,)),
4     keras.layers.Dense(1) # saída
5 ])
```

Listing 4: Uma MLP minúscula em Keras (só para fixar a ideia).

5.3 Função de perda: medir o erro

Definição 5.3 (Função de perda). Número que mede o quão erradas estão as previsões. Treinar = ajustar os pesos para *minimizar* a perda. Para prever números, duas perdas comuns são:

$$\text{MSE} = \frac{1}{n} \sum_i (\hat{y}_i - y_i)^2 \quad (\text{erro quadrático médio}),$$

$$\text{MAE} = \frac{1}{n} \sum_i |\hat{y}_i - y_i| \quad (\text{erro absoluto médio}).$$

Intuição 5.4 (MSE vs. MAE). O MSE eleva o erro ao quadrado, então pune *desproporcionalmente* os desvios grandes — um único ponto muito fora domina a conta. O MAE trata todos os desvios em pé de igualdade, sendo mais **robusto a outliers**.

5.4 Como a rede aprende: gradiente descendente

A perda depende dos pesos. Imagine a perda como a altitude de um terreno montanhoso e os pesos como sua posição: queremos descer até o vale. O **gradiente** aponta a direção de subida mais íngreme; então damos passos na direção *oposta*. O tamanho do passo é a **taxa de aprendizado** (*learning rate*).

- **Época** (*epoch*): uma passada por todos os dados de treino.
- **Lote** (*batch*): subconjunto processado por vez antes de atualizar os pesos.
- **Otimizador**: a receita de como dar os passos. Usamos o **Adam**, que ajusta o passo automaticamente e é o padrão moderno.

5.5 Overfitting, validação e *early stopping*

Definição 5.5 (Overfitting). Quando a rede *decora* o treino (inclusive seu ruído) em vez de aprender o padrão geral. Sinal: erro baixíssimo no treino, mas alto em dados novos.

Para detectar isso, separamos uma fatia do treino como **validação** e monitoramos a perda nela durante o treino. Enquanto a validação melhora, a rede aprende; quando ela começa a piorar (mesmo com o treino ainda melhorando), começou o overfitting. O **early stopping** interrompe o treino nesse ponto e restaura os melhores pesos. O **dropout** é outra defesa: durante o treino, desliga aleatoriamente uma fração dos neurônios, forçando a rede a não depender de detalhes específicos.

No NeuraTrade. Treinamos com a perda **MSE**, otimizador **Adam** (taxa 0,001), lotes de 32, **dropout** de 0,2 e **early stopping** com paciência de 10 épocas (restaurando os melhores pesos). Um detalhe vital para série temporal: a validação usa o *bloco final* do treino (não uma amostra aleatória) e **shuffle=False**, para não embaralhar o tempo e recriar o vazamento da Seção 4. Na detecção (Seção 8) preferimos o **MAE** como medida de erro, por sua robustez a *outliers*.

6 Séries temporais e redes recorrentes (LSTM)

6.1 Por que uma MLP comum não basta

Uma MLP trata cada entrada como um conjunto de números *sem ordem*. Mas em uma série temporal a **ordem é a informação**: “subiu três dias e caiu no quarto” é diferente de “caiu e subiu três”. Precisamos de uma rede que tenha *memória* da sequência.

6.2 Redes recorrentes (RNN)

Definição 6.1 (RNN). Uma rede recorrente processa a sequência passo a passo, carregando um **estado** (uma “memória”) que é atualizado a cada novo elemento. Assim, a saída no dia t depende não só de x_t , mas de tudo que veio antes.

RNNs simples têm um problema: ao processar sequências longas, o sinal de aprendizado ou *some* (gradiente que desaparece) ou *explode*, e a rede “esquece” o que viu há muitos passos.

6.3 LSTM: memória com portões

A **LSTM** (*Long Short-Term Memory*) resolve isso com uma célula de memória controlada por **portões** (*gates*) — pequenas sub-redes que decidem, a cada passo:

- **esquecer**: o que descartar da memória antiga;
- **entrada**: o que da informação nova guardar;
- **saída**: o que da memória expor como resultado do passo.

Intuição 6.2. Pense na LSTM como um leitor com um caderno de anotações. A cada frase ele decide o que apagar, o que anotar de novo e o que usar agora. Esse controle ativo da memória deixa a LSTM lembrar de padrões relevantes por muito mais tempo que uma RNN simples — ideal para capturar regimes de volatilidade que se estendem por semanas.

6.4 Janelas deslizantes: fatiando o tempo

A LSTM recebe *sequências de tamanho fixo*. Convertemos a série contínua em pedaços com a técnica da **janela deslizante**: uma janela de comprimento L varre a série de uma em uma posição.

Exemplo 6.3. Série $[a, b, c, d, e]$ com janelas de tamanho 3 vira: $[a, b, c]$, $[b, c, d]$, $[c, d, e]$. Cada janela é uma “frase” de 3 dias que a LSTM lê inteira.

```
1 import numpy as np
2 def make_windows(valores, L=30, passo=1):
3     idx = range(0, len(valores) - L + 1, passo)
4     janelas = np.stack([valores[i:i+L] for i in idx])
5     return janelas[..., np.newaxis]    # forma (n_janelas, L, 1)
```

Listing 5: Gerando janelas de tamanho fixo a partir de um vetor 1D.

A forma final $(n_janelas, L, 1)$ é o que a LSTM espera: n exemplos, cada um com L passos no tempo e 1 variável por passo (o log-retorno).

No NeuraTrade. Usamos janelas de **30 passos** (`window_size=30`) — cerca de seis semanas úteis —, valor consagrado em implementações de referência de detecção de anomalias com LSTM. As janelas são geradas *dentro* de cada partição (treino e teste separadamente), para que nenhuma janela cruze a fronteira temporal e reintroduza vazamento. Uma consequência (importante na avaliação): como as janelas se sobrepõem, um único dia anômalo aparece em até 30 janelas consecutivas — uma anomalia vira uma “rajada” de marcações.

7 Autoencoder: aprender a normalidade comprimindo

7.1 A ideia: comprimir e reconstruir

Definição 7.1 (Autoencoder). Uma rede que tenta copiar a própria entrada na saída, mas é obrigada a passar por um gargalo. Tem duas partes:

- **Encoder**: comprime a entrada em poucos números — o **código** ou *bottleneck* (gargalo).
- **Decoder**: tenta reconstruir a entrada original a partir desse código comprimido.

A perda é o quão diferente a reconstrução ficou da entrada (erro de reconstrução).

Intuição 7.2. É como pedir a alguém para resumir um texto em uma frase e depois reescrever o texto a partir só da frase. Para o resumo permitir uma boa reconstrução, ele precisa capturar a *essência* — os padrões que se repetem. Detalhes idiossincráticos se perdem no gargalo.

7.2 Por que isso detecta anomalias

Aqui está o truque central do projeto. Treinamos o autoencoder **só com dados normais**. Ele se torna especialista em reconstruir o normal — e *apenas* o normal, porque foi só isso que viu.

- Entrada **normal** $\Rightarrow$ reconstrução boa $\Rightarrow$ erro **baixo**.
- Entrada **anômala** (padrão que ele nunca aprendeu) $\Rightarrow$ reconstrução ruim $\Rightarrow$ erro **alto**.

O erro de reconstrução vira o nosso medidor de anomalia. Não precisamos rotular nada: a própria incapacidade de reconstruir denuncia o anômalo. É isso que torna o método *não supervisionado*.

Exemplo 7.3 (Fora de finanças). Um autoencoder treinado só com fotos de gatos aprende a reconstruir gatos com fidelidade. Mostre-lhe a foto de um carro: a reconstrução sai borrada e errada (erro alto). Sem nunca ter recebido o rótulo “isto não é gato”, ele acusa o estranho pela qualidade da reconstrução.

7.3 A arquitetura do NeuraTrade

Encadeamos LSTMs nas duas pontas, para respeitar a natureza sequencial dos dados:

```
1 from tensorflow import keras
2 L = 30 # passos por janela
3 modelo = keras.Sequential([
4     # ENCODER: comprime a janela em um vetor latente
5     keras.layers.LSTM(64, input_shape=(L, 1)),
6     keras.layers.Dropout(0.2),
7     keras.layers.Dense(16, activation="relu"), # gargalo (latent_dim=16)
8     # DECODER: reconstrói a janela de 30 passos
9     keras.layers.RepeatVector(L),
10    keras.layers.LSTM(64, return_sequences=True),
11    keras.layers.Dropout(0.2),
12    keras.layers.TimeDistributed(keras.layers.Dense(1)),
13 ])
14 modelo.compile(optimizer="adam", loss="mse")
```

Listing 6: LSTM-Autoencoder (esqueleto da arquitetura do projeto).

Definição 7.4 (Dimensão latente (*latent_dim*)). O tamanho do gargalo. Pequeno demais: a rede não consegue nem reconstruir o normal (perde sinal). Grande demais: o gargalo deixa de comprimir e a rede pode aprender a copiar *qualquer* coisa — inclusive anomalias —, perdendo poder de detecção. É um equilíbrio.

No NeuraTrade. Gargalo de **16** (`latent_dim=16`), LSTMs de 64 unidades, *dropout* 0,2. Testamos 8/16/32 e a perda de validação ficou *plana* entre eles — ou seja, o resultado é pouco sensível nessa faixa —, o que confirmou 16 como escolha equilibrada (compressão de $\approx 4:1$ sobre a janela de 30 passos). Treina-se **um modelo por ação**, para que cada um aprenda a “normalidade” específica do seu ativo e setor.

8 Detecção: do erro ao alarme

Temos um medidor de anomalia (o erro de reconstrução). Falta a regra que transforma um número contínuo em uma decisão “anômalo ou não”. Essa regra é o **limiar** (*threshold*).

8.1 Erro de reconstrução por janela

Para cada janela de 30 passos, comparamos a reconstrução com a entrada e resumimos a diferença em um número — usamos o **MAE** (Seção 5), robusto a *outliers*:

```
1 import numpy as np
2 recon = modelo.predict(X)           # X: (n_janelas, 30, 1)
3 erro = np.mean(np.abs(recon - X), axis=(1, 2)) # um valor por janela
```

Listing 7: Erro de reconstrução (MAE) por janela.

8.2 Limiar estático: uma linha fixa

Definição 8.1 (Limiar estático). Um valor fixo, calculado como o **percentil 95 do erro no treino**. Qualquer janela de teste com erro acima dele é marcada como anômala.

Calculá-lo *no treino* (e não no teste) evita vazamento: o limiar reflete a normalidade aprendida, não o período sob avaliação.

Intuição 8.2. É como definir “febre” por um termômetro travado em 37,8°C. Simples e estável — mas não se adapta. Se a pessoa naturalmente vive mais quente por um período (mudança de regime), o termômetro fixo apita o tempo todo.

8.3 Limiar dinâmico: uma linha que se adapta

Definição 8.3 (Limiar dinâmico). O percentil 95 recalculado continuamente sobre uma **janela móvel** dos erros recentes (252 dias úteis ≈ 1 ano). Acompanha o regime local de volatilidade.

O ponto delicado é a **causalidade**: a janela móvel só pode usar o *passado*. Se inclísse o próprio ponto (ou o futuro), uma anomalia ajudaria a definir o limiar que deveria detectá-la — vazamento de novo. Garantimos a causalidade com um deslocamento de uma posição (`shift(1)`): cada ponto é avaliado contra a estatística dos dias *anteriores* a ele.

```
1 import pandas as pd
2 s = pd.Series(erro)
3 # shift(1) exclui o proprio ponto -> janela usa apenas o passado
4 limiar_din = s.shift(1).rolling(252, min_periods=252).quantile(0.95)
```

Listing 8: Limiar dinâmico causal (só passado), com pandas.

No NeuraTrade. Comparamos os dois limiares (uma das contribuições do projeto). No AMER3, após a fraude, o erro fica permanentemente alto: o estático marca $\approx 27\%$ das janelas (perde utilidade), enquanto o dinâmico cai para $\approx 14\%$ ao tratar o novo patamar como o “normal recente”. Em PETR4/VALE3 (estáveis) ambos ficam em $\approx 3\%$. Estudamos ainda o tamanho da janela móvel: 126 dias fica ruidoso, 504 super-suaviza (marca quase nada), e **252** é o equilíbrio.

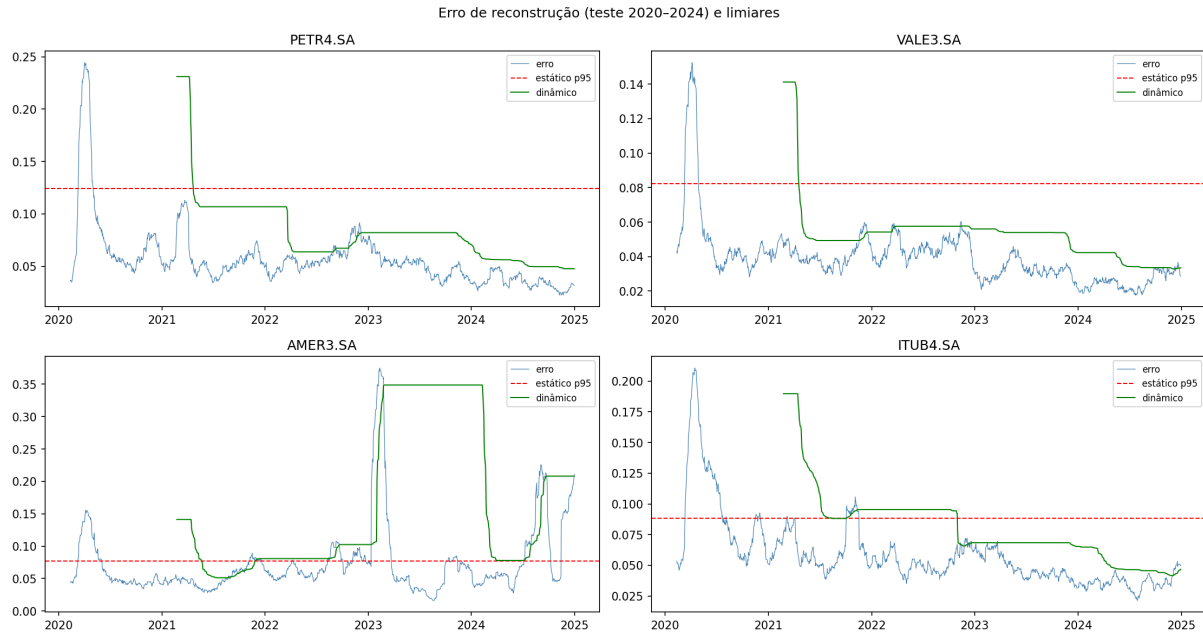


Figura 1: Erro de reconstrução no teste (2020–2024) com o limiar estático (tracejado vermelho) e o dinâmico (verde). Onde o regime muda muito (AMER3), o estático “satura” e marca quase tudo; o dinâmico se recalibra. Em ativos estáveis os dois quase coincidem.

9 Avaliação: como saber se está funcionando

9.1 O dilema: não temos gabarito

Por ser não supervisionado, não temos uma lista oficial de dias anômalos para comparar. Como medir acerto sem gabarito? Criamos um **gabarito artificial**.

9.2 Injeção sintética de anomalias

Definição 9.1 (Injeção sintética). Inserimos, de propósito, perturbações (*choques*) em *posições conhecidas* da série de teste. Como sabemos exatamente onde colocamos cada choque, podemos verificar se o detector os encontra.

```
1 import numpy as np
2 rng = np.random.default_rng(42) # seed fixa -> reprodutivel
3 pos = rng.choice(len(serie), size=50, replace=False)
4 perturbada = serie.copy()
5 perturbada[pos] += magnitude # o "choque"
6 gabarito = np.zeros(len(serie), int); gabarito[pos] = 1
```

Listing 9: Injetar choques em posições aleatórias (gera o gabarito).

9.3 As métricas: precisão, revocação e F1

Comparando o que o detector marcou com o gabarito, cada janela cai em uma de quatro caixas (a **matriz de confusão**):

	É anomalia (gabarito)	Não é
Detector marcou	VP (verdadeiro positivo)	FP (falso positivo)
Detector não marcou	FN (falso negativo)	VN (verdadeiro negativo)

Definição 9.2 (Precision, Recall, F1).

$$\text{Precision} = \frac{VP}{VP + FP} \quad (\text{dos que marquei, quantos eram mesmo?}),$$

$$\text{Recall} = \frac{VP}{VP + FN} \quad (\text{das anomalias reais, quantas peguei?}),$$

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (\text{média harmônica das duas}).$$

Intuição 9.3 (Exemplo médico). Um teste de doença. **Recall** alto: pega quase todos os doentes (poucos escapam) — mas pode alarmar gente saudável. **Precision** alta: quando alarma, quase sempre acerta — mas pode deixar passar casos. Quase sempre há um *trade-off*: aumentar um tende a baixar o outro. O **F1** resume os dois em um número, punindo desequilíbrios (se um for baixo, o F1 cai).

```
1 from sklearn.metrics import precision_score, recall_score, f1_score
2 p = precision_score(gabarito_janela, marcado_janela, zero_division=0)
3 r = recall_score(gabarito_janela, marcado_janela, zero_division=0)
4 f1 = f1_score(gabarito_janela, marcado_janela, zero_division=0)
```

Listing 10: Métricas com scikit-learn.

9.4 Duas armadilhas que aprendemos na prática

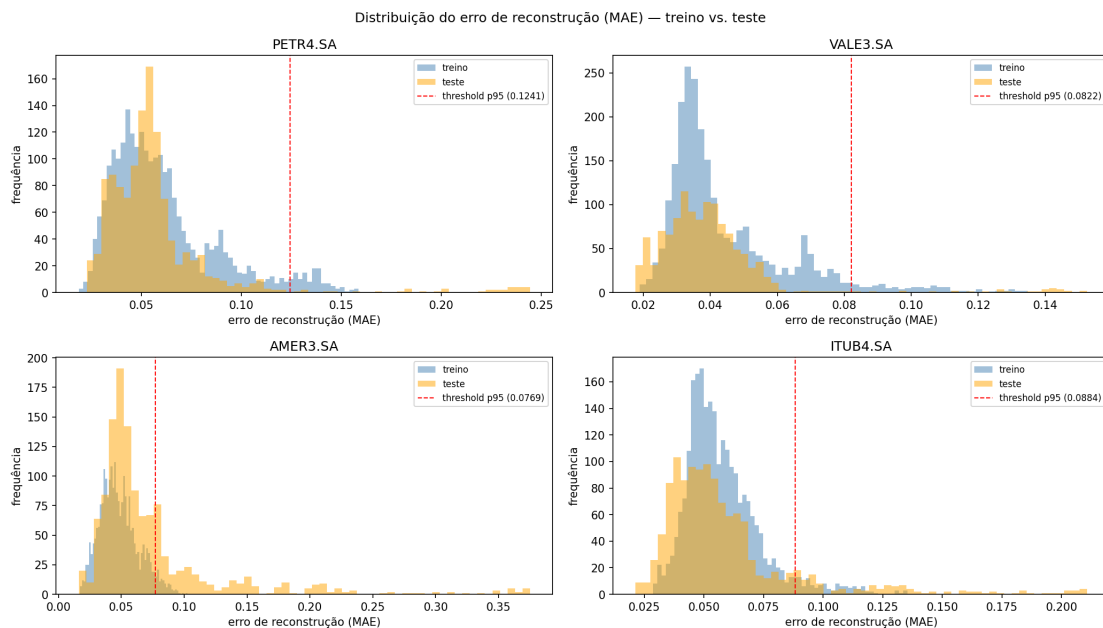


Figura 2: Distribuição do erro de reconstrução (treino vs. teste) com o limiar p95. A cauda à direita do limiar concentra as janelas marcadas; o deslocamento treino→teste é bem maior no AMER3.

- **Taxa-base.** Como o p95 já marca $\approx 5\%$ das janelas por construção (Seção 3), há um piso de falsos positivos que limita a *precision* máxima. Sempre se reporta isso junto.
- **Diluição por janela.** O erro é a *média* sobre 30 passos. Um choque de um único dia contribui só $\approx 1/30$ do erro da janela — logo é facilmente “afogado”. Por isso o *recall* de choques pontuais é naturalmente baixo, sobretudo em ativos calmos.

Definição 9.4 (Calibração relativa $k\sigma$). Em vez de um choque de tamanho absoluto (igual para todos), usamos um choque de k desvios-padrão do próprio ativo: $\text{mag} = k\sigma$. Assim “um choque de 4σ ” tem o mesmo significado estatístico em todas as ações, tornando a dificuldade comparável entre elas.

No NeuraTrade. Injetamos 50 choques por ativo e medimos P/R/F1 para os dois limiares. Descoberta honesta e instrutiva: mesmo calibrando em 4σ , o *recall* permanece baixo nos ativos estáveis — não por culpa do choque, mas pela **diluição por janela** e pela altura do limiar. Conclusão: a avaliação sintética *valida o procedimento de medição*, não entrega um número final de desempenho. Caminhos futuros: choques de vários dias ou usar o *máximo* (em vez da média) do erro na janela.

10 Validação narrativa e comparação setorial

A avaliação sintética (Seção 9) é controlada porém artificial. Falta o teste de realidade: **as anomalias detectadas coincidem com eventos que de fato abalaram o mercado?**

10.1 Cruzar com a história

Montamos uma linha do tempo curada de eventos econômicos e políticos brasileiros de 2010 a 2024 (crash da COVID, fraude da Americanas, Brumadinho, intervenção na Petrobras, *impeachment*, etc.). Cada evento é classificado como:

- **sistêmico:** afeta todo o mercado (ex.: COVID);
- **idiossincrático:** específico de uma empresa (ex.: fraude da Americanas → só AMER3).

Definição 10.1 (Tolerância de *matching*). Dizemos que um evento foi “coberto” se houve ao menos uma anomalia detectada dentro de ± 30 dias dele. A tolerância de 30 dias não é arbitrária: é o próprio tamanho da janela (`window_size`), a granularidade natural da detecção. Datas de eventos também são âncoras aproximadas (o anúncio e a reação do mercado podem diferir em dias).

10.2 Os dois testes que isso permite

- **Contágio (sistêmico):** um evento que abala tudo deve acender anomalia em *todos* os ativos. A Figura 3 mostra o crash da COVID disparando nos quatro, de setores distintos.
- **Especificidade (idiossincrático):** um evento de uma empresa deve acender *só* nela. A fraude da Americanas isola-se em AMER3; Brumadinho, em VALE3.

A combinação dos dois mostra que o modelo, treinado por ativo, captura tanto choques amplos de mercado quanto problemas específicos — o objetivo de “generalização cruzada entre setores” do projeto.

Intuição 10.2 (Por que cobertura “só” de 50% não é ruim). A cobertura agregada de eventos fica em torno de 50%, mas o número cru engana: o detector é calibrado para a *cauda* (5% maiores erros), então deliberadamente ignora eventos de impacto difuso e dispara nos mais fortes. O que importa é o *casamento certo*: AMER3 cobre 9 de 10 dos seus eventos. Um detector que acendesse em tudo teria 100% de cobertura e nenhuma utilidade — de novo, o trade-off precision/recall da Seção 9.

No NeuraTrade. Como há *uma* ação por setor ($n = 1$), a comparação setorial é um **estudo de caso qualitativo**, não inferência estatística: diferenças podem refletir a empresa, não o setor. Reportamos isso explicitamente para não exagerar nas conclusões.

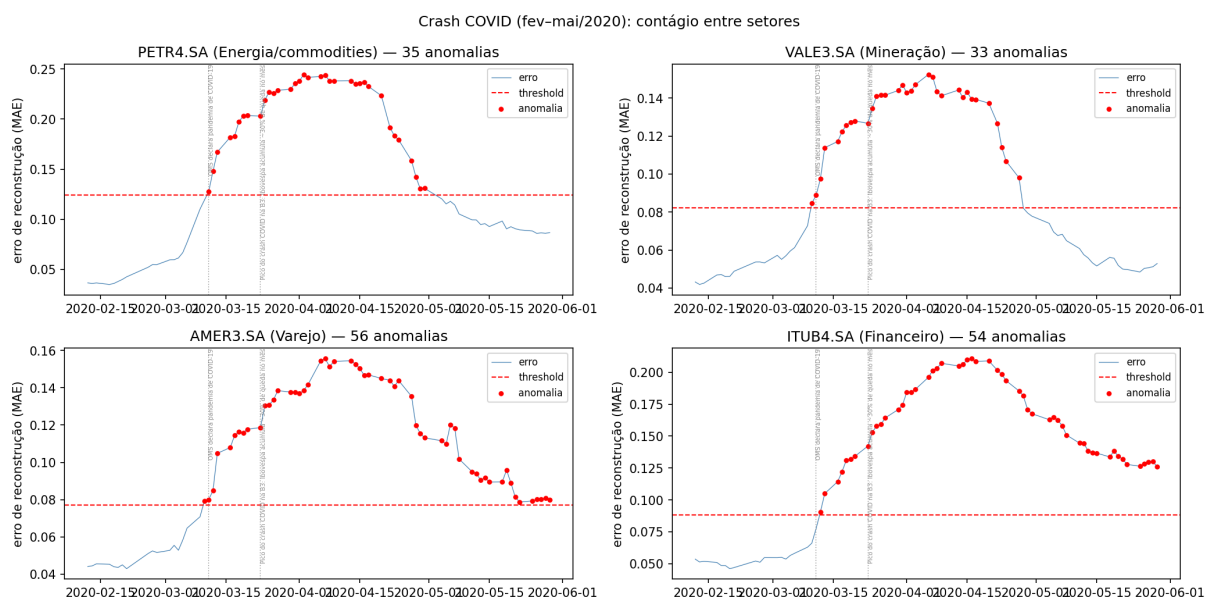


Figura 3: Crash da COVID (fev-mai/2020): os quatro ativos, de setores diferentes, sinalizam anomalias concentradas no mesmo período — evidência de contágio sistêmico.

11 Mapa: decisão → teoria → registro

Cada decisão de projeto está registrada como um *Architecture Decision Record* (ADR) em docs/adr/. A tabela abaixo liga cada ADR à seção teórica deste guia que o fundamenta, servindo de índice de estudo.

11.1 O fio condutor

Vale reter a corrente lógica que une tudo:

1. Preço → **log-retorno** (estaciona e compara) [2].
2. Split temporal + normalização só no treino (**sem vazamento**) [4].
3. Fatiar em **janelas** de 30 passos para a LSTM [6].
4. **LSTM-Autoencoder** aprende a reconstruir o normal [7].
5. **Erro de reconstrução** alto = anomalia; limiar por **percentil** [8].
6. Avaliar com **injeção sintética** e P/R/F1; cruzar com **eventos reais** [9, 10].

Cada elo existe para proteger o seguinte de uma falha conceitual — a maioria delas, alguma forma de espiar o futuro.

12 Fase 2: três aprimoramentos e sua teoria

Esta seção explica, no mesmo espírito didático do guia, a teoria por trás das três evoluções planejadas para a segunda fase do projeto (milestone M8, registradas em ADR-0009, 0010 e 0011). São *propostas*: a teoria já está madura, a implementação virá depois.

12.1 Agregar o erro: média esconde o pico

O modelo produz um erro de reconstrução para *cada* um dos 30 passos de uma janela. Para decidir se a janela é anômala, precisamos resumir esses 30 números em **um**. Esse passo de resumo chama-se *agregação* (ou *pooling*).

Tabela 1: Decisões do projeto, o conceito que as sustenta e onde estudá-lo.

ADR	Decisão	Base teórica (seção)
0001	Split temporal <i>antes</i> da normalização; scaler só no treino	Vazamento temporal (4); normalização (3)
0002	Janela de 30 passos (<code>window_size</code>)	Janelas deslizantes (6)
0003	Arquitetura LSTM-Autoencoder, <code>latent_dim=16</code> , perda MSE	Autoencoder (7); LSTM (6); perdas (5)
0004	Treino: Adam, <i>early stopping</i> , <code>shuffle=False</code>	Aprendizado e overfitting (5); vazamento (4)
0005	Limiar estático p95 + limiar dinâmico causal (252)	Percentil (3); detecção e causalidade (8)
0006	Avaliação por injeção sintética; calibração $k\sigma$; setorial	Métricas e matriz de confusão (9)
0007	Coleta, <code>auto_adjust</code> , tratamento do AMER3	Preço, <i>splits</i> e log-retorno (2)
0008	Linha do tempo de eventos; tolerância $\pm \text{window_size}$	Validação narrativa e setorial (10)

Intuição 12.1. Imagine 30 dias, 29 calmos e 1 com um choque enorme. Se você tira a **média** dos erros, o pico de 1 dia é dividido por 30 — vira uma marola. Se você tira o **máximo**, o pico sobrevive intacto. A média é um detector de “a janela toda está estranha”; o máximo é um detector de “algum dia desta janela está muito estranho”.

Formalmente, dado o vetor de erros por passo $e_1, \dots, e_{30}$ de uma janela:

$$\text{erro}_{\text{mean}} = \frac{1}{30} \sum_{t=1}^{30} e_t, \quad \text{erro}_{\text{max}} = \max_t e_t, \quad \text{erro}_p = \text{percentil}_p(e_1, \dots, e_{30}).$$

O máximo é sensível ao melhor sinal, mas também ao *ruído* de um único dia atípico — pela mesma razão que rejeitamos o “máximo do erro de treino” como limiar (Seção 8). O percentil alto (p.ex. p90) é o meio-termo: pega quase o pico, sem depender de um só ponto.

No NeuraTrade. Trocar a média pelo máximo/percentil **aumenta o recall** de choques de um dia. Mas o limiar p95 do projeto foi calculado sobre a *média* do erro: como o máximo muda toda a distribuição, o limiar precisa ser **recalibrado** sobre o novo escore — senão a taxa de falsos positivos explode. Medido nos quatro ativos (notebook 07), o *máximo* dobrou o *recall* (de 0,16 para 0,35) e ainda *umentou* a *precision* (de 0,55 para 0,84): o pico de um dia, antes diluído, fica nítido no pior passo. Decisão em ADR-0009.

12.2 Validação que respeita o tempo: *walk-forward*

Para escolher um hiperparâmetro (como a dimensão do gargalo, `latent_dim`) queremos uma estimativa *confiável* de quão bem cada valor generaliza. Uma única partição de validação dá uma estimativa “sortuda ou azarada”. A validação cruzada repete a medição em vários cortes e tira a média.

Intuição 12.2. A validação cruzada clássica (*k-fold*) embaralha os dados e usa pedaços aleatórios para validar. Em série temporal isso é **proibido**: embaralhar deixaria o modelo treinar em dias *futuros* para prever dias *passados* — espiar o futuro (Seção 4). O *walk-forward* corrige isso:

sempre treina no passado e valida no futuro adjacente, avançando como uma janela que “anda para frente”.

Exemplo 12.3. Com `TimeSeriesSplit` e 4 folds sobre 2010–2019: treina em 2010–11 valida 2012; treina em 2010–12 valida 2013; e assim por diante. A fatia de treino *cresce*, a de validação é sempre o bloco seguinte. Nunca o futuro entra no treino.

No NeuraTrade. A regra de ouro do projeto (normalizar só com o treino, ADR-0001) vale **dentro de cada fold**: o `MinMaxScaler` é reajustado a cada corte, usando apenas o treino daquele fold, e as janelas são geradas dentro de cada partição. Custa K vezes mais treino, mas dá média $\pm$ desvio do erro por candidato. Rodado (notebook 08, com 10 folds — a convenção de Kohavi, 1995) para `latent_dim` $\in \{8, 16, 32\}$, os três praticamente empataram (`val_loss` $\approx 0,0154$): o gargalo **16 foi confirmado** e, de quebra, o modelo mostrou-se *insensível* ao tamanho do gargalo nessa faixa — um sinal de robustez. Decisão em ADR-0010.

12.3 Mais de uma variável: entrada multivariada (OHLCV)

Até aqui a entrada é *univariada*: só o log-retorno do fechamento. Mas cada dia de bolsa tem cinco números (OHLCV): abertura, máxima, mínima, fechamento e **volume** (quantas ações foram negociadas). O volume costuma *anteceder* o movimento de preço — um sinal precursor que a entrada univariada joga fora.

Intuição 12.4. Pôr cinco variáveis juntas parece sempre melhor, mas surge um problema de **escala**: o volume está na casa dos milhões ($\sim 10^7$) e o log-retorno na casa dos centésimos ($\sim 10^{-2}$). Se normalizar tudo junto, o volume “engole” o retorno e a perda só enxerga volume. Por isso normalizamos **coluna a coluna**: cada variável ganha sua própria escala $[0, 1]$.

Há ainda dois cuidados. Primeiro, o volume **cresce ao longo dos anos** (não é estacionário): o range de 2010–2019 não cobre os volumes de 2020+, e o `MinMaxScaler` satura tudo em 1.0 — aplicamos `log1p` ao volume antes de escalar para comprimir essa faixa. Segundo, Open/High/Low/Close são quase a mesma informação (correlação $\sim 99\%$): por isso começamos por **Close+Volume** (30, 2) e só vamos a OHLCV (30, 5) se valer a pena.

No NeuraTrade. Com a entrada multivariada o erro vira um *vetor* (um por canal). Guardar o erro **por canal** permite dizer se a anomalia foi de *preço* ou de *volume* — e é o que finalmente habilita a injeção de `volume_spike`, antes impossível por falta do canal de volume (Seção 9). Treinado e testado (notebook 09, Close+Volume): ao injetar um pico de volume, o erro do **canal de volume sobe** (de +0,05 a +0,36) e o do canal de preço fica em ≈ 0 — a anomalia é atribuída corretamente ao volume. Decisão em ADR-0011.

12.4 Fecho da Fase 2 (M9)

Os três experimentos viraram decisão. A agregação **max** foi testada nos dados reais (não só nos sintéticos) e passou a ser o *default*: marcou a mesma fração de janelas que a média ($\approx 10\%$), sem “ver anomalia em tudo”. O **OHLCV completo** foi testado e *descartado* — acrescentar Abertura/Máxima/Mínima ao Close+Volume não melhorou (e às vezes piorou) a reconstrução; ficamos no par Close+Volume. E o tamanho do gargalo (`latent_dim`) mostrou-se *insensível* também no modelo multivariado. Lição geral: medir é barato e às vezes contraria nossa intuição — aqui, a suposta redundância do OHLC não era tão forte quanto se pensava, mas o resultado ainda assim recomendou a simplicidade.

13 Glossário

Anomalia

Comportamento que destoa do padrão normal aprendido. Aqui: janela cujo erro de reconstrução supera o limiar.

Autoencoder

Rede que comprime a entrada em um gargalo e a reconstrói; o erro de reconstrução denuncia o que é estranho.

Batch (lote)

Subconjunto de exemplos processado antes de atualizar os pesos.

Bottleneck (gargalo)

Camada estreita do autoencoder que força a compressão; seu tamanho é a *latent_dim*.

Causalidade

Propriedade de usar apenas informação do passado; essencial para não vazar o futuro no limiar dinâmico.

Desvio-padrão (σ)

Medida do espalhamento típico dos dados em torno da média; base da volatilidade.

Dropout

Desligar neurônios ao acaso durante o treino para combater *overfitting*.

Early stopping

Parar o treino quando a perda de validação para de melhorar, restaurando os melhores pesos.

Época (epoch)

Uma passada completa por todos os dados de treino.

Erro de reconstrução

Diferença (MAE ou MSE) entre a janela de entrada e a reconstruída; o medidor de anomalia.

F1

Média harmônica de *precision* e *recall*.

Gradiente descendente

Algoritmo que ajusta os pesos descendo a “encosta” da função de perda.

Janela deslizante

Recorte de comprimento fixo que varre a série, gerando as sequências de entrada da LSTM.

Log-retorno

$\ln(P_t/P_{t-1})$; variação relativa que soma no tempo e é imune a *splits*.

LSTM

Rede recorrente com portões que controlam uma memória de longo prazo, boa para sequências.

MAE / MSE

Erro absoluto médio / erro quadrático médio; o MSE pune mais os desvios grandes, o MAE é mais robusto.

Normalização Min–Max

Reescala linear para $[0, 1]$ usando mínimo e máximo (aprendidos só no treino).

Overfitting

Decorar o treino em vez de generalizar.

Percentil (p95)

Valor abaixo do qual cai dada fração dos dados; o p95 deixa 5% acima.

Precision

Dos itens marcados como anomalia, a fração que era anomalia de fato.

Recall

Das anomalias reais, a fração que o detector encontrou.

Regime

Estado prolongado do mercado (calmo ou turbulento) caracterizado pelo nível de volatilidade.

Seed

Semente do gerador aleatório; fixá-la torna os resultados reprodutíveis.

Taxa-base

Fração de marcações inevitável por construção do limiar (aqui $\approx 5\%$ pelo p95); limita a *precision* máxima.

Vazamento (leakage)

Uso indevido de informação do futuro, que infla artificialmente o desempenho medido.

Volatilidade

Intensidade da oscilação dos retornos; medida pelo desvio-padrão.